

GitHub 入门指南（零基础版）

适用人群：完全没接触过 Git 和 GitHub 的新手 **环境：**Windows 11 + Git Bash（macOS / Linux 大部分命令通用） **目标：**从零开始，带你完成「注册账号 → 装好工具 → 上传第一个项目 → 日常使用 → 参与协作」的全过程

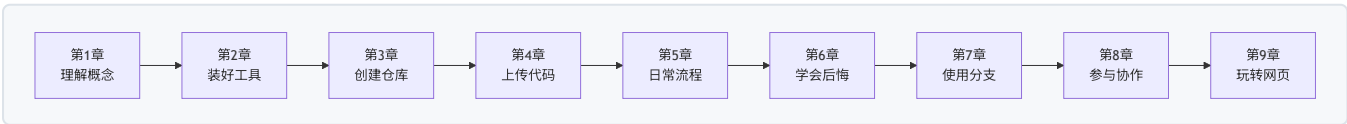
这份指南怎么用

- 全程**零基础假设**：每一步都会告诉你「点哪里、输什么、为什么」
- 每章末尾有**小任务**，跟着动手做一遍比读十遍都管用
- 第 10 章是**速查表 + 常见报错对照表**，出问题时优先去那里查
- 文中所有命令都可以在 **Git Bash** 里直接复制粘贴（注意：命令里的 `你的用户名`、`邮箱` 等要换成你自己的）

章节地图

章节	内容	你会学到
第 1 章	Git 与 GitHub 是什么	搞懂基本概念，建立整体认识
第 2 章	安装与注册准备	装好 Git、注册 GitHub、配好免密登录
第 3 章	创建你的第一个仓库	在网页上创建仓库，认识仓库页面
第 4 章	把代码上传到 GitHub	把本地项目推送到云端（核心技能）
第 5 章	日常开发工作流	每天改代码的标准流程
第 6 章	版本回退与撤销	各种「后悔药」，改错了也不怕
第 7 章	分支入门	用分支安全地开发新功能
第 8 章	协作与 Pull Request	和别人一起写代码的正确姿势
第 9 章	GitHub 网页功能导航	Issues、Actions、Pages 等实用功能
第 10 章（附录）	命令速查与常见问题	速查表 + 报错解决对照表

学习路线建议



- **只想快速用起来：**看第 2、3、4 章即可

- **想系统掌握**：按顺序通读，每章练习都做一遍
 - **遇到问题**：直接翻第 10 章对照表
-

开始吧！翻到第 1 章，花 10 分钟搞懂 GitHub 到底是什么。

第 1 章 Git 与 GitHub 是什么

1.1 先讲一个大家都遇到过的痛苦

写代码（或者写文档、做设计）时，你是不是经常遇到这些情况：

- 改了几百行代码，第二天发现改坏了，**却回不去了**
- 想试试新想法，又怕弄乱现有代码，于是复制出 项目-副本、项目-最终版、项目-最终版2
- 和同学/同事合作，两人改了同一个文件，**后保存的把先保存的覆盖了**
- 电脑硬盘坏了 / 文件误删，**几个月的心血没了**

这些问题都有一个共同的解决方案：**版本控制**。

1.2 Git 是什么

Git 是一个「版本控制工具」，安装在你自己的电脑上，专门负责记录文件的每一次修改。

可以把它想象成两个东西的结合：

1. **游戏存档系统**：每完成一段工作就「存个档」，以后任何时刻都可以读档回到过去
2. **云盘的历史版本功能**：但不是只记一个文件，而是记录整个项目文件夹的所有变化

Git 的核心能力：

能力	说明
记录历史	每次「提交」都是一个存档点，带时间、作者、说明
随时回退	任何存档点都可以恢复回来
分支开发	开一条独立「平行线」做实验，不影响主线
合并改动	把不同人的改动安全地合在一起，自动检测冲突

注意：Git 和「编程」没有必然关系。写论文、做设计稿、写小说，任何文本文件都可以用 Git 管理版本。

1.3 GitHub 是什么

GitHub 是一个「网站」 (<https://github.com>)，专门用来**存放 Git 仓库**，让代码（或其他文件）可以：

- **备份在云端**：电脑丢了代码还在
- **多人协作**：大家把代码推到同一个地方，互相拉取
- **公开分享**：全世界有上亿开发者在这里开源代码，你可以免费学习和使用
- **项目管理**：提需求 (Issues)、做审查 (Pull Request)、跑自动化 (Actions)

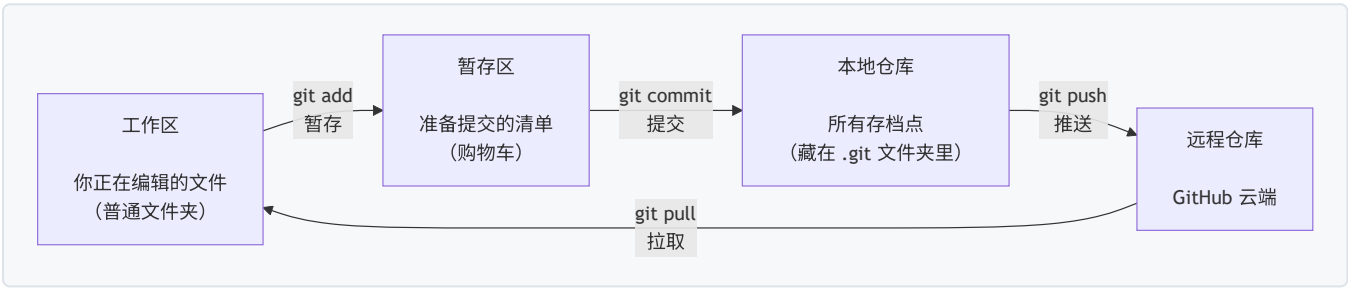
一句话总结它们的关系：

名称	是什么	装在哪里
Git	版本控制 工具 （软件）	你的电脑上
GitHub	存放 Git 仓库的 网站	云端

Git 是工具，GitHub 是平台。Git 不会因为你不用 GitHub 而失去价值，但没有 Git，GitHub 无从谈起。类似的平台还有 GitLab、Gitee（国内），原理都一样，学会 GitHub 之后全部通用。

1.4 你的代码在几个「地方」之间流动

理解下面这张图，就理解了 Git 工作的核心原理：



四个「地方」，三种角色：

区域	通俗理解	你在里面做什么
工作区	你正在编辑的文件	正常写代码、改文件
暂存区	购物车	挑好这次要「存档」的文件
本地仓库	你电脑上的存档记录	存放所有历史版本
远程仓库	GitHub 云端	备份 + 与别人同步

很多新手的第一道坎就是「为什么不能一步到位，非要 add 又 commit？」——因为 Git 允许你**挑选**要保存的内容：这次改了 5 个文件，但只想先存档其中 2 个，暂存区就是干这个用的。养成习惯后你会爱上这个设计。

1.5 必须掌握的 10 个名词

名词	英文	通俗解释
仓库	Repository (Repo)	一个项目的「档案室」，包含所有文件 + 所有历史
提交	Commit	一次存档，包含改动内容、作者、时间、说明
分支	Branch	一条独立的开发线，类似「平行宇宙」

名词	英文	通俗解释
主分支	main	默认的主线，通常存放稳定代码
远程	Remote	云端仓库的地址，默认叫 <code>origin</code>
克隆	Clone	把云端仓库完整复制一份到本地
推送	Push	把本地的新存档上传到云端
拉取	Pull	把云端的新存档下载到本地
合并	Merge	把一条分支的改动并入另一条分支
拉取请求	Pull Request (PR)	请求别人把你的改动合并进主线（协作核心）

1.6 一个典型的使用场景

假设你和两个朋友合作一个课程设计：

1. 你把项目创建成 GitHub 仓库，**push** 上去
2. 朋友 A 和 B 各自 **clone** 到电脑上开发
3. A 写好了登录功能，**push** 上去，并发起一个 **Pull Request**
4. 你和 B 在网页上**审查**代码，没问题就点**合并**
5. B 每天开工前先 **pull** 一下，同步别人的改动
6. 答辩前一周，你们用 **分支** 并行开发「演示 PPT」和「修 bug」，互不干扰

整个流程没有一次「文件传来传去」，没有一次「覆盖了别人的代码」。这就是 Git + GitHub 的日常。

1.7 本章小任务

- [] 打开 <https://github.com> 随便逛逛，感受一下「世界上最大的代码托管平台」
- [] 找到你听过的开源项目（比如 VS Code: github.com/microsoft/vscode），点一下右上角的 **Star** 收藏它

下一章：装好 Git、注册 GitHub 账号，把准备工作一次性做完。

第 2 章 安装与注册准备

本章目标：注册好 GitHub 账号、装好 Git、配置好身份和免密登录。只做一次，以后不用再管。

2.1 注册 GitHub 账号

1. 打开 <https://github.com>
2. 点击右上角 **Sign up** (注册)
3. 依次填写：
 - **Email**: 你的邮箱 (QQ 邮箱、163、Gmail 都可以)
 - **Password**: 密码 (至少 8 位)
 - **Username**: 用户名 —— 这是你未来主页地址 `github.com/你的用户名` 的一部分
4. 可能让你做一个图片验证 (证明你是人类)，照做即可
5. 到邮箱里找到 GitHub 发来的验证邮件，点击验证链接

用户名怎么起？

- 只能包含字母、数字、连字符 -
- 别人会用 `github.com/用户名` 访问你的主页，**未来会写在简历上**，建议起得正式一些
- 例： `zhangsan-dev` (推荐)、 `zhangsan_dev_2024` (不推荐，下划线看起来不正式)

2.2 安装 Git (Windows)

方式一：官网下载安装 (推荐新手)

1. 打开 <https://git-scm.com/downloads>
2. 网页会自动识别你的系统，点击 **Download** 下载安装包
3. 双击运行安装包，一路点 **Next** 保持默认设置即可
 - 遇到选择默认编辑器的页面 (Choosing the default editor)，保持默认的 Vim 就行，本书不涉及
 - 其余选项全部默认，不用改任何东西
4. 安装完成

方式二：命令行一键安装

按 `Win + X` → 打开「终端 (管理员)」，输入：

```
winget install --id Git.Git -e --source winget
```

验证安装是否成功

在桌面或任意文件夹**右键**，如果看到菜单里有 **Git Bash Here**，说明装好了。点击它打开 Git Bash，输入：

```
git --version
```

看到类似 `git version 2.45.2.windows.1` 的输出，就成功了。

Git Bash 是什么？ 它是一个模拟 Linux 命令行的窗口。本书所有命令都在它里面输入。从今天起，把它当作你的 Git 操作台。

2.3 配置你的身份（必须做！）

每次提交（存档）都会带上「作者是谁」的信息，GitHub 靠它把你的提交记录和你的账号头像关联起来。**不配置就无法提交。**

在 Git Bash 中依次输入（把引号里的内容换成你自己的）：

```
git config --global user.name "张三"
git config --global user.email "zhangsan@qq.com"
```

要点：

- `--global` 表示对这台电脑上所有仓库生效，只需配置这一次
- **邮箱必须和 GitHub 注册邮箱一致**，否则提交记录不会关联到你的头像
- 名字可以写英文昵称，也可以写中文

验证一下：

```
git config --list
```

输出里能看到 `user.name=张三` 和 `user.email=zhangsan@qq.com` 就对了。

2.4 配置免密登录（SSH 密钥）

每次 push/pull 都要输密码很烦，而且现在 GitHub 已经不支持纯密码推送了。**配置一次 SSH 密钥，以后推送永远不用输密码。**

SSH 密钥是一对「锁和钥匙」：私钥放在你电脑里，公钥放在 GitHub 上，配对成功即证明你是你。

第 1 步：生成密钥

在 Git Bash 中输入（邮箱换成你的 GitHub 邮箱）：

```
ssh-keygen -t ed25519 -C "zhangsan@qq.com"
```

然后：

1. 提示 Enter file in which to save the key —— **直接按回车**（使用默认位置）
2. 提示 Enter passphrase —— **直接按回车**（不设密码）
3. 再次提示确认 —— **再按一次回车**

看到类似下面的图案（一堆随机字符组成的长方形），就生成好了：

```
The key's randomart image is:
+--[ED25519 256]--+
|      .o+      |
|      ...      |
+----[SHA256]-----+
```

第 2 步：复制公钥

```
cat ~/.ssh/id_ed25519.pub
```

输出一行以 `ssh-ed25519` 开头的长字符串。**用鼠标选中整行，右键复制**（Git Bash 里复制是选中后按回车或右键）。

常见错误：复制时漏掉了开头或结尾。公钥必须**完整一行**，从 `ssh-ed25519` 开始到邮箱结束。

第 3 步：把公钥粘贴到 GitHub

1. 打开 GitHub 网站，点右上角**你的头像** → **Settings**（设置）
2. 左侧菜单找到 **SSH and GPG keys**，点进去
3. 点绿色按钮 **New SSH key**
4. **Title** 随便填，比如「我的笔记本」（给自己看的备注）
5. **Key type** 保持默认 `Authentication Key`
6. 把刚才复制的公钥**粘贴**到 **Key** 大输入框里
7. 点 **Add SSH key**

第 4 步：测试连接

回到 Git Bash 输入：

```
ssh -T git@github.com
```

第一次连接会问：


```
Are you sure you want to continue connecting (yes/no)?
```

输入 `yes` 回车。之后如果看到：

```
Hi zhangsan! You've successfully authenticated, but GitHub does not provide shell access.
```

恭喜，免密登录配置完成！ 后面的 `Hi 你的用户名` 就代表成功了（最后的 `does not provide shell access` 是正常提示，不是错误）。

常见失败排查

报错	原因	解决
<code>Permission denied (publickey)</code>	公钥没添加成功	回到第 3 步重新复制粘贴，注意完整性
一直连不上、超时	网络问题	公司/校园网可能屏蔽了 22 端口，改用 HTTPS 方式克隆即可（见 3.4 节）
生成密钥时报错	命令输错	检查 <code>-t ed25519</code> 中间的空格

2.5 本章检查清单

全部打勾才算准备好：

- ☐ GitHub 账号注册完成，能登录
- ☐ `git --version` 能输出版本号
- ☐ `git config --list` 能看到自己配置的名字和邮箱
- ☐ `ssh -T git@github.com` 显示 `Hi 你的用户名!`

下一章：在 GitHub 网页上创建你的第一个仓库。

第 3 章 创建你的第一个仓库

本章目标：在 GitHub 网页上创建第一个仓库，认识仓库页面的每个部分。

3.1 什么是仓库 (Repository)

仓库 = 一个项目的完整档案室，里面装着：

- 项目的所有文件和文件夹
- 所有的修改历史（谁、什么时候、改了什么）
- 协作相关的资料（问题、讨论、文档）

一个仓库通常对应一个项目。你可以在 GitHub 上拥有无数个仓库，**免费的私有仓库**也不限量。

3.2 公开 (Public) 还是私有 (Private) ？

类型	谁能看到	适合
Public 公开	全世界的任何人都能看	开源项目、学习作品集、想让别人复用的代码
Private 私有	只有你和被你邀请的人	课程作业、公司项目、不想公开的实验

不确定就选 **Private**，以后随时可以在 Settings 里改成 Public，反之亦然，没有次数限制。

3.3 动手：创建仓库（一步步来）

1. 登录 GitHub，点右上角的 + 号，选择 **New repository**（新仓库）
2. 填写创建表单：

表单项	怎么填
Owner	保持你的用户名（如果你加入了组织，可以选组织名）
Repository name	起个名字，如 <code>my-first-repo</code> （用英文和连字符，不要用中文和空格）
Description	一句话描述（可选）： 我的第一个仓库，用来练习 <code>Git</code>
Public / Private	选 Private （练习用，不想公开）
Add a README file	勾选 ！这会自动创建一份项目说明文件

3. 点击绿色按钮 **Create repository**，仓库创建完成！

3.4 认识仓库主页

创建完成后，你会看到仓库主页，从上到下依次是：

区域	作用
顶部仓库名	用户名 / 仓库名，旁边有 Public/Private 标记
功能标签栏	Code （代码）、 Issues （问题）、 Pull requests （合并请求）、 Actions （自动化）、 Settings （设置）等
文件列表	你的所有文件。现在应该只有一个 <code>README.md</code>
About 栏（右侧）	项目简介、网址、标签
绿色 Code 按钮	最常用的按钮 ，用来获取克隆地址

绿色 Code 按钮的三种地址

点一下 **Code**，你会看到三个标签：

- **HTTPS**：形如 `https://github.com/用户名/仓库名.git`，走网页协议
- **SSH**：形如 `git@github.com:用户名/仓库名.git`，走密钥认证（第 2 章已配置）——**推荐**
- **GitHub CLI**：给 GitHub 命令行工具用的

因为第 2 章配置了 SSH，之后克隆请都用 **SSH** 标签下的地址。

3.5 README.md 是什么

README 是仓库的「门面」：打开仓库主页，正文显示的就是它。

- 介绍项目是干什么的、怎么安装、怎么使用
- 文件名必须是 `README.md`（.md 表示 Markdown 格式，一种轻量标记语言，用 `#` 表示标题、`-` 表示列表）
- 求职时，面试官点开你的仓库第一眼看到的就是它，**务必认真写**

现在就试试：在仓库主页点击 `README.md` 文件名 → 点右上角**铅笔图标**（Edit）→ 改成：

```
# 我的第一个仓库

这是我的 GitHub 学习记录。

## 学习进度

- [x] 注册 GitHub 账号
- [x] 创建第一个仓库
- [ ] 上传第一个项目
```

改完后，滚动到底部，点绿色的 **Commit changes**（提交修改）按钮。

注意：这是**在网页上直接编辑**，改动直接保存到仓库，绕过了本地 Git。适合改 README 这种小事，但真正的代码开发不要这样做——后面章节会讲正规流程。

3.6 三种开始使用仓库的方式

拿到一个新仓库（不管是刚建的还是别人的），有三种用法，对应后面三章：

方式	场景	对应章节
A. 本地已有代码，推到新仓库	代码已经在电脑里了	第 4 章
B. 克隆仓库到本地开发	仓库在云端，要下载到电脑改	第 5 章开头
C. 网页上直接编辑	只改改文档、README	本章 3.5 节

3.7 本章小任务

- [] 创建了属于自己的第一个仓库（Public 或 Private 都行）
- [] 在仓库主页找到了 Code / Issues / Settings 的位置
- [] 点开 Code 按钮，看到了 HTTPS 和 SSH 两种地址
- [] 用网页编辑器修改了 README.md 并提交

下一章：把电脑里的代码上传到这个仓库（方式 A，最核心的技能）。

第 4 章 把代码上传到 GitHub

本章目标：把电脑里已有的项目文件夹，上传到 GitHub 仓库。**这是 GitHub 使用中最重要的一项技能**，请务必跟着做一遍。

4.1 场景设定

假设你电脑里有一个项目文件夹 `D:\code\my-website`，里面有一些网页文件：

```
my-website/  
├─ index.html  
├─ style.css  
└─ images/  
    └─ logo.png
```

现在你想把它传到 GitHub 上：既能备份，又能分享给别人。**总共 6 条命令，逐条来。**

4.2 第 1 步：进入项目文件夹

右键 `my-website` 文件夹，选 **Git Bash Here**（或在 Git Bash 里用 `cd` 命令进入）。

Git 的很多命令只对「当前所在的文件夹」生效，所以每次操作前都要确认自己**在项目文件夹里**。

4.3 第 2 步：初始化本地仓库

```
git init
```

作用：把这个普通文件夹变成「Git 管理的文件夹」。执行后：

- 文件夹里会悄悄生成一个隐藏的 `.git` 文件夹（存所有历史记录的地方）
- 看到输出 `Initialized empty Git repository...` 就是成功

如果看不到 `.git` 文件夹：Windows 资源管理器默认隐藏以点开头的文件，勾选「查看 → 隐藏的项目」就能看到。**不要手动删它**，删了历史就没了。

4.4 第 3 步：查看当前状态

```
git status
```

你会看到类似输出：

```
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html
    style.css
    images/
```

解读：

- `No commits yet`：还没有任何存档
- `Untracked files`（未跟踪文件）：Git 发现这几个文件，但还没开始记录它们——它们现在就像「门口排队但没进档案室」

`git status` 是最常用的命令，**任何时候不知道自己在哪，先敲它**。它会告诉你：有哪些新文件、改了哪些文件、下一步该干什么。

4.5 第 4 步：暂存文件

```
git add .
```

作用：把当前文件夹里的**所有**文件加入「暂存区」（购物车）。最后的 `.` 表示「当前文件夹的全部内容」。再敲一次 `git status`，变化如下：

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   index.html
    new file:   style.css
    new file:   images/logo.png
```

`Changes to be committed` = 这些文件已在购物车里，随时可以结账（提交）。

只想暂存某个文件？

```
git add index.html
```

4.6 第 5 步：提交（存档）

```
git commit -m "首次提交：网站初始版本"
```

作用：把暂存区的内容正式存档，`-m` 后面引号里是这次存档的**说明文字**（提交信息）。

成功后输出类似：

```
[master (root-commit) 8f3a2b1] 首次提交：网站初始版本
3 files changed, 128 insertions(+)
```

- `8f3a2b1` 是这次提交的编号（提交 ID，以后回退时用）
- `3 files changed` 表示 3 个文件发生了变化

提交信息怎么写？ 给未来的自己（和协作者）看：说明这次「做了什么」。对比：

```
git commit -m "aaa" # ❌ 毫无信息量
git commit -m "新增首页和样式文件，完成页面框架" # ✅ 一看就懂
```

新手常见错误：`git commit` 忘了加 `-m`，会进入一个奇怪的 Vim 编辑界面出不来。**解决办法：按 Esc，输入 `:q!` 回车退出，重新用带 `-m` 的命令提交。**

4.7 第 6 步：关联远程仓库

先到 GitHub 网页上，按第 3 章的方法创建一个**空仓库**（这次**不要勾选** Add a README file——两边都有内容会导致推送冲突），然后：

```
git branch -M main
git remote add origin git@github.com:你的用户名/仓库名.git
```

逐条解释：

- `git branch -M main`：把本地主分支改名为 `main`（Git 旧版本默认叫 `master`，GitHub 现在统一用 `main`，改一下保持一致）
- `git remote add origin 地址`：告诉本地仓库「我的云端在哪个地址」，并给这个地址起个别名叫 `origin`（起源）。以后说 `origin` 就指你的 GitHub 仓库

地址从哪复制？ 仓库主页 → 绿色 Code 按钮 → **SSH** 标签 → 复制那一串 `git@github.com:...` 地址。

验证关联是否成功：

```
git remote -v
```

应输出两行：

```
origin  git@github.com:你的用户名/仓库名.git (fetch)
origin  git@github.com:你的用户名/仓库名.git (push)
```

4.8 第 7 步：推送到 GitHub

```
git push -u origin main
```

作用：把本地仓库的所有存档上传到云端。`-u` 是「记住这次对应关系」，**之后日常推送只需敲 `git push` 三个单词**。

成功的输出类似：

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (5/5), 2.1 KiB | 2.1 MiB/s, done.
To github.com:你的用户名/仓库名.git
* [new branch]      main -> main
```

去 GitHub 网页刷新仓库页面——你的文件全部出现在云端了！

4.9 完整流程回顾

六条命令，记住这个流程：

git init
初始化仓库

git status
查看状态

git add .
加入暂存区

git commit -m 说明
提交存档

git branch -M main
改分支名

git remote add origin 地址
关联云端

git push -u origin main
推送到云端

 文件出现在 GitHub 上

4.10 本节常见报错

报错	原因	解决
<code>fatal: not a git repository</code>	不在项目文件夹里执行命令	右键项目文件夹选 Git Bash Here 再执行
<code>error: remote origin already exists</code>	重复执行了 <code>remote add</code>	用 <code>git remote set-url origin 新地址</code> 修改, 或不管它直接 <code>push</code>
<code>! [rejected] main -> main (fetch first)</code>	云端已有本地没有的内容 (比如建仓库时勾了 README)	先 <code>git pull origin main --allow-unrelated-histories</code> , 解决提示的冲突后再 <code>push</code>
<code>Permission denied (publickey)</code>	SSH 密钥没配好	回第 2 章 2.4 节重配, 或改用 HTTPS 地址 (需用 Personal Access Token 当密码)
<code>Everything up-to-date</code>	本地没有新提交	正常提示, 说明云端已经是最新的

4.11 本章小任务

- [] 找一个自己电脑里的小项目 (或新建一个文件夹写几个文件), 按 4.2 ~ 4.8 完整走一遍
- [] 在 GitHub 网页上确认文件都传上去了
- [] 改一个文件, 再执行一遍 `git add .` → `git commit` → `git push`, 在网页上看到更新

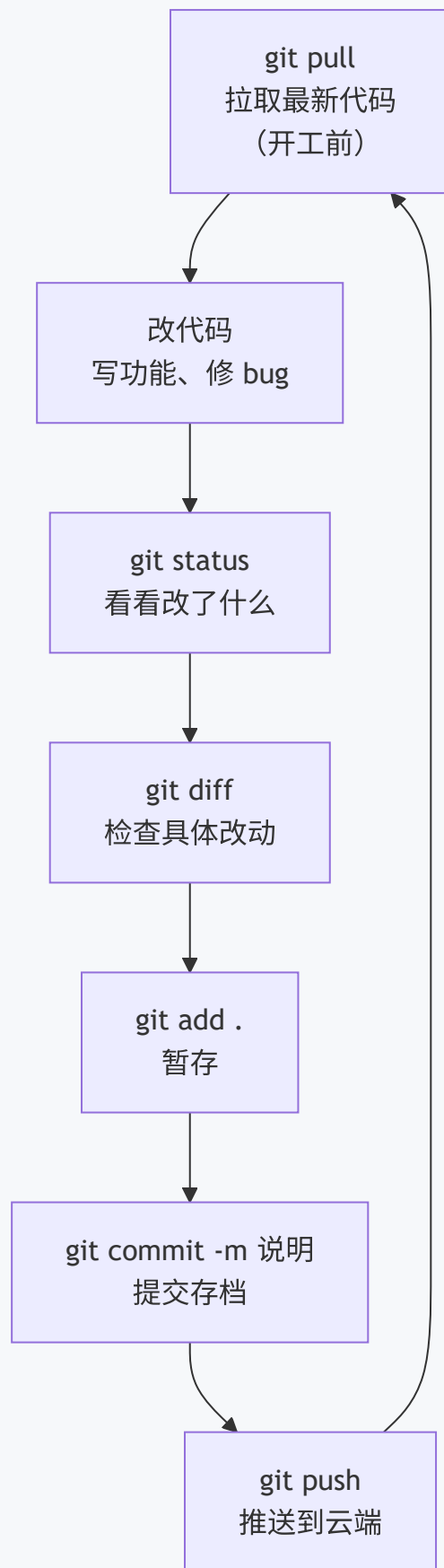
下一章: 学会每天的标准工作节奏——拉取、修改、提交、推送的完整循环。

第 5 章 日常开发工作流

本章目标：掌握「每天打开电脑写代码」的标准流程，形成肌肉记忆。

5.1 每日循环：拉取 → 修改 → 提交 → 推送

日常开发就是一个不断重复的循环：



- **一个人开发**时，循环可以简化：改代码 → add → commit → push (pull 和 diff 可以偶尔做)
- **多人协作**时，每一步都不能省，尤其是开工前的 pull

5.2 开工前：拉取最新代码

```
git pull
```

作用：把云端的最新内容同步到本地。**想象你和队友同时编辑一份文档：你开工前必须先拿到队友昨晚的改动，否则你们会在两个版本上各自修改。**

- 一个人开发：pull 只是把别的电脑上推的内容同步过来，没有也无妨
- 多人开发：**每天开工第一件事就是 pull**，避免「各自改旧版」

输出示例：

```
Already up to date.          # 云端没有新东西
```

或者：

```
Updating 8f3a2b1..3c7d9e5
Fast-forward
 index.html | 4 +--
 1 file changed, 2 insertions(+), 2 deletions(-)
```

5.3 改完代码后：查看状态

```
git status
```

这是你「改完一轮代码」后第一个要敲的命令。三种典型输出：

输出	含义	下一步
Untracked files: 新文件	新建的文件，Git 还不认识	git add 加入跟踪
Changes not staged for commit: modified: 文件名	已跟踪的文件被修改了	git add 暂存
Changes to be committed:	文件已暂存，等待提交	直接 git commit

阅读习惯：输出里有英文提示语，用翻译软件查一次，以后就都认识了。

5.4 检查具体改了什么

```
git diff
```

作用：显示每个文件**具体哪些行**被改动了。输出中：

- 以 `-` 开头（红色）：被删除的旧内容
- 以 `+` 开头（绿色）：新增的内容

`git diff` 只显示「工作区 vs 暂存区」的差异。暂存之后想看差异，用 `git diff --staged`。对新手来说，**提交前用 diff 自查一遍**，能避免把调试代码、临时文件误提交。

5.5 暂存

```
git add .           # 暂存当前文件夹所有改动
git add 文件名       # 只暂存某个文件
git add 文件1 文件2  # 暂存多个文件
```

场景建议：

- 改动都相关（都是同一个功能的）→ `git add .` 一步到位
- 改了 5 个文件但只想提交 2 个 → 逐个 `git add`，其余文件留在工作区

5.6 提交

```
git commit -m "说明文字"
```

提交信息规范（强烈建议从第一天就养成习惯）：

前缀	含义	示例
feat:	新功能	feat: 新增用户注册接口
fix:	修 bug	fix: 修复登录按钮点击无响应
docs:	文档	docs: 更新 README 安装说明
refactor:	重构（不改功能）	refactor: 提取公共函数
style:	格式调整	style: 统一缩进为 4 空格

完整格式：类型：一句话说明做了什么

```
git commit -m "fix: 修复登录按钮点击无响应，原因是事件未绑定"
```

原则：**一个提交只做一件事**。不要攒了三天杂七杂八的改动一次性提交——将来想回退某一步时会非常痛苦。

5.7 推送

```
git push
```

因为第 4 章用了 `-u` 建立关联，日常推送就这么简单。完成后去 GitHub 网页刷新，能看到最新的提交。

5.8 查看历史

```
git log --oneline          # 一行一条的精简历史
git log --oneline --graph  # 带分支线的图形化历史
```

输出示例：

```
3c7d9e5 (HEAD -> main, origin/main) fix: 修复导航栏错位
8f3a2b1 feat: 新增首页
5b1c0d4 首次提交: 网站初始版本
```

- 最上面是最新的提交
- 每行开头那串字符是提交 ID，第 6 章「回退」要用
- `HEAD` 表示你当前所在的位置

5.9 一个完整的工作日示例

假设你 9:00 开工，写一个「用户登录功能」：

```
# 9:00 开工，同步队友昨晚的改动
git pull

# 9:05 ~ 11:00 写代码：新建 login.html，修改 index.html 加登录入口
# 11:00 查看状态
git status
# 输出: login.html 是新文件，index.html 被修改

# 11:02 检查改动没问题
git diff

# 11:05 暂存并提交
git add .
git commit -m "feat: 新增用户登录页面，并在首页加入入口"

# 11:06 推送到云端
git push

# 14:00 下午继续：发现登录页有个 bug，修好后
git add login.html
git commit -m "fix: 修复登录表单密码框无法输入的问题"
git push
```

一天的工作 = 若干次「add → commit → push」的小循环 + 开工时的一次 pull。

5.10 只改了一半、突然要切去做别的事怎么办

```
git stash      # 把当前未提交的改动「藏起来」，工作区恢复干净
# ... 去做别的事，做完提交回来 ...
git stash pop  # 把藏起来的改动取回来继续改
```

常见场景：改到一半，突然要紧急修一个 bug。stash 能把半成品暂时收进抽屉，等修完 bug 再取出来继续。

5.11 本章小任务

- [] 按 5.9 的节奏完整走一个「小循环」：pull → 改文件 → status → diff → add → commit → push
- [] 用 `git log --oneline` 看到自己今天的所有提交
- [] 用 `git stash` 藏一次改动，再用 `git stash pop` 取回来

下一章：各种「后悔药」——改错了、提交错了，如何安全地回退。

第 6 章 版本回退与撤销（后悔药大全）

本章目标：搞懂「改错了怎么办」。按「错误发生的位置」分四层，一层比一层重，逐层认识。

6.1 后悔药地图

错误可能发生在四个层级，各有各的解药：



6.2 第 ① 层：从暂存区拿出来（最安全）

场景：`git add .` 手快了，把调试文件也加进了购物车。

```
git restore --staged 文件名
```

再看 `git status`，该文件回到 `Changes not staged` 状态。

- **影响**：只是「从购物车拿出来」，文件内容**一个字都不会少**
- 想全部拿出来：`git restore --staged .`

6.3 第 ② 层：丢弃工作区修改（内容会丢！）

场景：今天的修改越改越乱，想彻底回到「最近一次提交」时的样子。

```
git restore 文件名      # 丢弃单个文件的修改
git restore .            # 丢弃所有文件的修改
```

⚠️ 这条命令会彻底删除你未提交的修改，无法找回。 执行前用 `git diff` 确认一下：这些改动确实不要了吗？

安全替代方案：先 `git stash` 把改动藏起来，后悔了还能 `git stash pop` 找回。

6.4 第③层：撤销提交（三种力度）

先明确概念：每次提交都有一个**提交 ID**，用 `git log --oneline` 查看（第一列就是）。撤销提交的本质是「把 HEAD 指针移回某个历史位置」。

命令	提交记录	暂存区	工作区改动	力度
<code>git reset --soft 提交ID</code>	撤销提交	改动留在暂存区	完整保留	最温柔
<code>git reset --mixed 提交ID</code>	撤销提交	改动回到未暂存	完整保留	默认力度
<code>git reset --hard 提交ID</code>	撤销提交	清空	彻底删除	最暴力

场景 A：提交信息写错了 → 改一改

```
git commit --amend -m "新的提交信息"
```

效果：用新信息覆盖最近一次提交的信息。如果 amend 时不加 `-m`，会进入编辑器修改。

场景 B：不该提交的也提交了 → 撤销提交但保留改动

```
git reset --soft HEAD~1
```

解释：

- `HEAD~1` = 「当前所在位置的上一次提交」（`~2` 就是上上次）
- `--soft` 撤销提交记录，改动**完整保留**在暂存区，重新 add/commit 即可

场景 C：要回到某个历史版本，后面的全部不要了

```
git log --oneline          # 先找到目标提交 ID，比如 8f3a2b1
git reset --hard 8f3a2b1
```

⚠ `--hard` 之后的提交和改动**全部消失**。只建议在单人开发、且确定要抛弃一切时使用。

补充：已推到云端，如何撤销

如果后悔的提交**已经 push 到 GitHub**：

- **单人仓库**：可以本地 `reset` 后 `git push --force`（强制覆盖云端）。多人仓库**禁止**这样做，会毁掉队友的工作
- **多人仓库**：用安全的 `git revert 提交ID` —— 它不删除历史，而是生成一次「反向操作」的新提交来抵消那次改动：

```
git revert 3c7d9e5
git push
```

6.5 后悔药速查表

我想.....	命令	风险
把文件从暂存区拿出来	<code>git restore --staged 文件</code>	无
丢弃对某文件的修改	<code>git restore 文件</code>	改动丢失
修改最近一次提交信息	<code>git commit --amend -m "新信息"</code>	无
撤销最近一次提交、保留改动	<code>git reset --soft HEAD~1</code>	无
回到某个历史版本、抛弃之后一切	<code>git reset --hard 提交ID</code>	高，无法找回
撤销已推送的提交（多人协作）	<code>git revert 提交ID + push</code>	无

6.6 实战演练：完整走一遍「犯错→后悔」

1. 在项目里随便改点内容，提交三次：

```
git add . && git commit -m "第一次修改"
# 再改点内容
git add . && git commit -m "第二次修改"
# 再改点内容
git add . && git commit -m "第三次修改"
```

2. 用 `git log --oneline` 看到 3 条新提交

3. 发现「第三次修改」不该要，撤销它但保留改动：

```
git reset --soft HEAD~1
git status      # 改动还在暂存区
```

4. 重新提交成正确的内容：

```
git commit -m "重新提交"
```

5. 体验丢弃工作区修改：

```
# 随便改一个文件
git restore 那个文件
git status      # 工作区干净了，改动没了
```

这个演练强烈建议亲手做一遍——**在练习中犯错，比在真实项目里犯错便宜得多。**

6.7 本章小任务

- [] 用 `git log --oneline` 找到自己最早的提交 ID
- [] 练习 `git commit --amend` 修改提交信息
- [] 练习 `git reset --soft HEAD~1` 撤销提交并确认改动还在
- [] 体会一次 `git restore 文件` 丢弃改动的「无法挽回感」

下一章：用分支实现「平行开发」——新功能、新实验，都不用怕弄坏主线。

第 7 章 分支入门

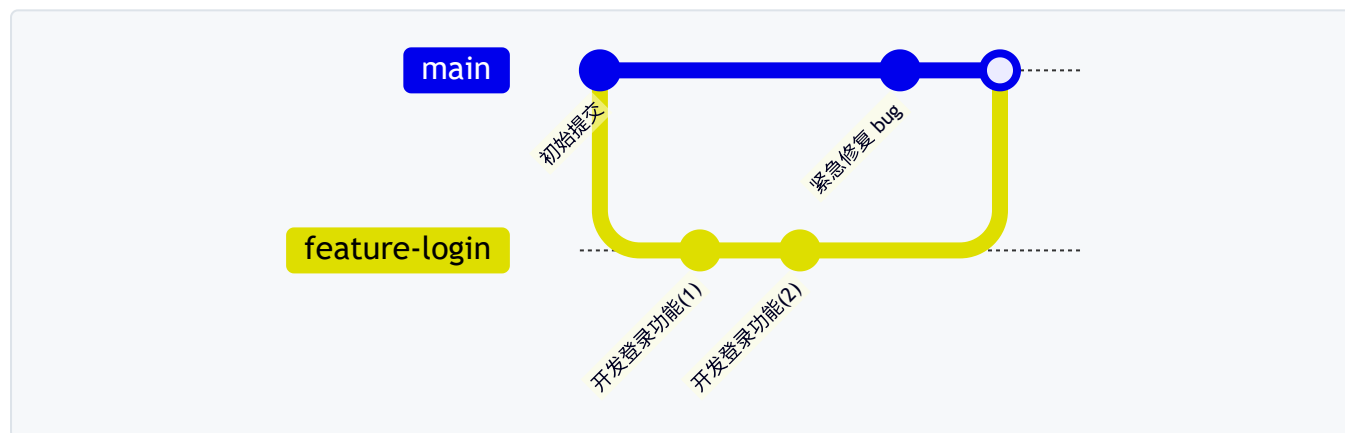
本章目标：理解分支的概念，掌握创建、切换、合并分支，以及解决合并冲突。

7.1 什么是分支

分支 = 一条独立的开发线，就像「平行宇宙」：

- 在分支上随便折腾，**不影响主线**
- 做成功了，把分支「合并」回主线
- 做失败了，把分支一删，主线毫发无伤

为什么要用分支？ 没有分支时，你想开发新功能，只能直接改主线代码——改到一半来个紧急 bug 要修，你会进退两难。有了分支：



主线上修 bug 和登录功能的开发**同时进行、互不干扰**，最后再合并。

7.2 分支的基本操作

查看分支

```
git branch
```

输出示例（带 * 的是当前所在分支）：

```
* main
  feature-login
```

创建分支

```
git branch 分支名
```

切换分支

```
git switch 分支名  
# 旧写法: git checkout 分支名 (效果相同, Git 教程里常出现, 认识即可)
```

创建并直接切换过去 (最常用)

```
git switch -c 分支名
```

命名习惯: 分支名用小写英文 + 连字符, 能看出用途。例: `feature-login`、`fix-navbar-bug`、`docs-update`。

删除分支

```
git branch -d 分支名    # 已合并的分支用 -d  
git branch -D 分支名    # 未合并也要强制删除用 -D (慎用)
```

注意: 不能删除**自己当前所在**的分支, 先 switch 到别的分支再删。

7.3 完整示例：开一条分支做新功能

```
# 1. 确认在 main 分支且代码最新
git switch main
git pull

# 2. 开新分支开发
git switch -c feature-dark-mode

# 3. 改代码、提交（分支上的提交与主线互不影响）
git add .
git commit -m "feat: 新增深色模式样式"

# 4. 开发完成，推送到云端（第一次推送要 -u）
git push -u origin feature-dark-mode

# 5. 回到 main 分支，把新功能合并进来
git switch main
git merge feature-dark-mode

# 6. 合并成功，删除分支（本地）
git branch -d feature-dark-mode
# 远程分支也删掉（如果不需要了）
git push origin --delete feature-dark-mode

# 7. 把合并结果推送到云端
git push
```

合并时输出 `Fast-forward` 表示「直接快进」（main 上没别的提交，直接把指针移过去）；如果 main 上也有了新提交，Git 会生成一个**合并提交**，两者都保留。

7.4 合并冲突：新手必经之路

冲突是怎么产生的

两人（或两个分支）修改了**同一文件的同一位置**，Git 无法判断听谁的，就会停下来说：**「你们打一架，谁赢了我再继续。」**

动手体验一次冲突（强烈建议亲手做）

第 1 步：制造冲突。 在 main 分支上，编辑 `test.txt` 写一行：

```
早餐吃豆浆
```

提交：

```
git add test.txt && git commit -m "main 上写豆浆"
```

第 2 步：开新分支，改同一行：

```
git switch -c conflict-demo
# 编辑 test.txt，把内容改成：
# 早餐吃油条
git add test.txt && git commit -m "分支上写油条"
```

第 3 步：回到 main 合并，冲突出现：

```
git switch main
git merge conflict-demo
```

输出：

```
Auto-merging test.txt
CONFLICT (content): Merge conflict in test.txt
Automatic merge failed; fix conflicts and then commit the result.
```

第 4 步：打开 test.txt，看到冲突标记：

```
<<<<<< HEAD
早餐吃豆浆
=====
早餐吃油条
>>>>>> conflict-demo
```

- <<<<<< HEAD 到 ===== 之间：**当前分支 (main) **的内容
- ===== 到 >>>>>> conflict-demo 之间：**被合并分支**的内容

第 5 步：手动编辑文件，删掉三行标记，保留想要的内容，比如决定两个都要：

```
早餐吃豆浆和油条
```

第 6 步：保存，告诉 Git 冲突已解决：

```
git add test.txt
git commit -m "解决冲突：早餐豆浆油条都吃"
```

提交冲突解决时**不要加 -m 以外的花活**，正常提交即可。提交后冲突就翻篇了。

小技巧：用 VS Code 打开冲突文件时，冲突区域会高亮，并提供 **Accept Current (接受当前)** / **Accept Incoming (接受对方)** / **Accept Both (都接受)** 按钮，一键解决，不用手删标记。

冲突解决后想反悔

```
git merge --abort
```

在提交冲突解决之前执行，可以取消本次合并，回到合并前的状态。

7.5 分支策略：日常怎么用分支

新手阶段记住两条最简单的规则：

分支	用途	纪律
main	只放「能跑、稳定」的代码	永远不直接在 main 上开发
feature-xxx	每个新功能 / 每次修复开一条	做完就合并回 main，然后删除

流程：main 上切出新分支 → 开发 → 合并回 main → 删分支。这套流程叫「功能分支工作流」，是业界最常用的工作流之一。

7.6 本章小任务

- [] 开一条分支，加一个新功能，合并回 main，删除分支
- [] 按 7.4 节亲手制造并解决一次冲突
- [] 试试 git merge --abort 取消一次合并

下一章：把你的分支用「Pull Request」交给别人审查合并——GitHub 协作的灵魂。

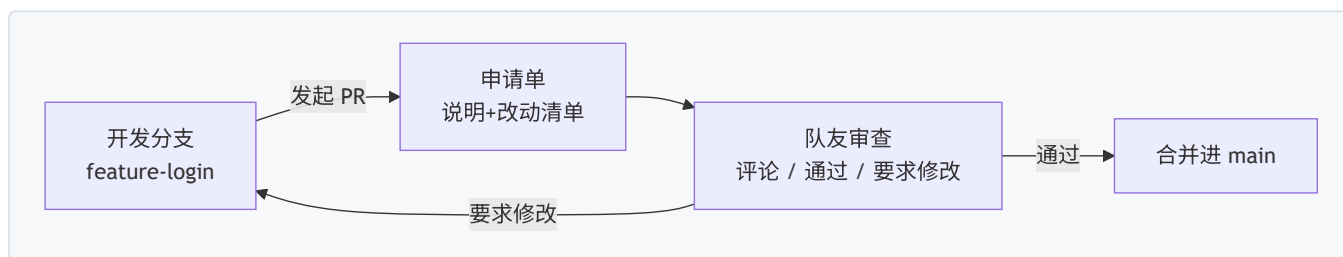
第 8 章 协作与 Pull Request

本章目标：学会 GitHub 协作的核心玩法——**Pull Request (拉取请求, 简称 PR)**，包括团队内部协作和给开源项目贡献代码两条路线。

8.1 Pull Request 是什么

PR = 一份「合并申请单」。

你在自己的分支上完成了开发，想合并进主分支。你不直接合并，而是发起一份申请：**「我的改动在这里，请求合并，请大家审查。」**



为什么不直接合并？ 因为：

- 别人的代码，你不敢随便动；**审查**是互相把关
- 大改动要有人**知情、讨论、拍板**
- 所有讨论记录都留在 PR 里，未来回看「当初为什么这么改」一目了然

8.2 路线一：同一个仓库里的团队协作

适用场景：一个团队共享一个仓库（比如小组课程设计、公司项目）。

完整流程（一步步来）

第 1 步：创建功能分支并开发

```
git switch main
git pull
git switch -c feature-login
# ... 开发、提交 ...
git push -u origin feature-login
```

第 2 步：在网页上发起 PR

1. 打开仓库主页，GitHub 通常会弹出一个黄色提示条：`feature-login had recent pushes`，点击旁边的 **Compare & pull request** 按钮（没有提示条就点 **Pull requests** 标签 → **New pull request**）
2. 确认合并方向：

- `base`: 要合并进的分支 → 选 `main`
- `compare`: 你开发的分支 → 选 `feature-login`

3. 填写申请单:

- **标题**: 一句话说明 (如 `feat: 新增用户登录功能`)
- **正文**: 写清楚三件事——**改了什么、为什么这么改、怎么测试**
- 右侧可以指定 **Reviewers** (请谁审查) 和 **Assignees** (谁负责)

4. 点 **Create pull request**

第 3 步: 队友审查 (Code Review)

审查者在 PR 页面的 **Files changed** 标签里逐行看改动:

- 在任意一行代码上点 **+** 号, 可以**行内评论**
- 看完点右上角 **Review changes**, 三选一:

选择	含义
Comment	只评论, 不表态
Approve	通过, 同意合并
Request changes	要求修改后才能合并

第 4 步: 按意见修改, PR 自动更新

审查提出意见后, 你在本地修改、提交、推送:

```
# 还在 feature-login 分支上
git add .
git commit -m "按审查意见: 修正变量命名"
git push
```

PR 页面会自动显示新提交, 无需重新发起。修改完在评论里回一句「已修改, 麻烦再看下」, 审查者重新 Approve。

第 5 步: 合并

全部通过后, 点 PR 底部的绿色 **Merge pull request** 按钮。

合并的三种方式

方式	效果	适合
Create a merge commit	生成一个合并提交, 保留所有分支提交	大团队, 想保留完整历史
Squash and merge	把 PR 里的所有提交 压缩成 1 个 再合并	新手推荐 , 主线历史干净
Rebase and merge	把你的提交「平移」到主线顶端, 历史呈一条直线	追求极致线性历史

团队里**统一一种方式**即可。个人项目强烈推荐 Squash and merge：你在功能分支上随意小步提交，合并进主线时只剩一条干净记录。

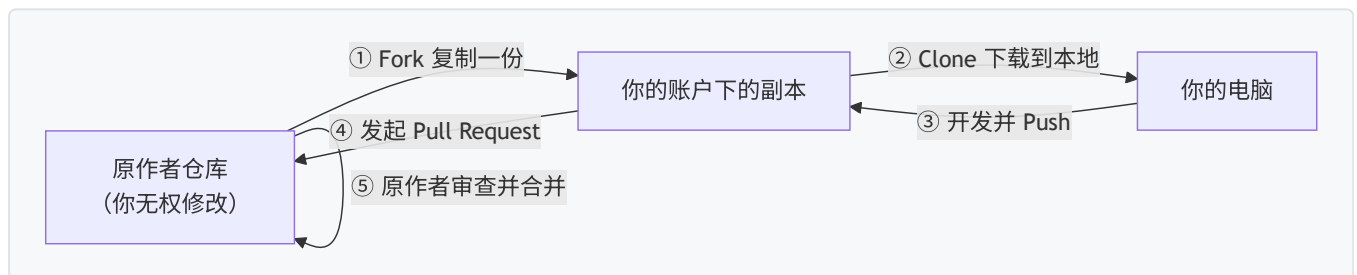
第 6 步：合并后清理

```
git switch main
git pull          # 同步合并后的 main
git branch -d feature-login # 删本地分支
git push origin --delete feature-login # 删远程分支（网页合并时勾了 delete branch 则不用）
```

8.3 路线二：给别人的开源项目贡献代码（Fork 流程）

适用场景：你想给一个**你没有写权限**的开源项目提改进（修 bug、加文档、加功能）。

核心思路：**你没有权限改别人的仓库，那就先复制一份到自己名下，改完再申请合回去。**这个「复制一份到自己名下」的操作叫 **Fork**。



完整流程

第 1 步：Fork

打开原作者仓库主页 → 点右上角 **Fork** 按钮 → **Create fork**。几秒后，你名下多了一个一模一样的仓库副本。

第 2 步：克隆你自己的副本到本地

```
git clone git@github.com:你的用户名/原作者仓库名.git
cd 原作者仓库名
```

第 3 步：添加原仓库为「上游」（upstream）

```
git remote add upstream git@github.com:原作者/原作者仓库名.git
git remote -v
```

输出应有 4 行：`origin` 指向**你的副本**（你推送到这里），`upstream` 指向**原作者仓库**（用来同步人家的更新）。

第 4 步：创建分支，开发，推送

```
git switch -c fix-typo
# ... 修改、提交 ...
git push -u origin fix-typo
```

第 5 步：发起 Pull Request

打开**你的副本**仓库主页 → 点 **Pull requests** → **New pull request** → 此时 base 会自动指向原作者仓库，确认无误 → 填写标题和说明 → **Create pull request**。

第 6 步：等待审查、按意见修改

和 8.2 节相同：原作者会评论、要求修改，你继续在 `fix-typo` 分支上提交推送，PR 自动更新。**被合并之后，你就正式成为这个开源项目的贡献者了**——这是简历上非常有含金量的一笔。

第 7 步（日常维护）：保持副本与上游同步

原作者仓库一直在更新，你的副本会渐渐落后，定期同步：

```
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

8.4 PR 的注意事项（提高被合并的概率）

给开源项目提 PR，被拒绝是家常便饭，但做好这几条能显著提高通过率：

- **先看 CONTRIBUTING.md**：很多项目规定了贡献规范（分支命名、提交信息格式），不遵守会直接被拒
- **先提 Issue 再提 PR**：大改动先在 Issues 里和作者讨论「该不该做」，达成共识再动手，避免白干
- **PR 越小越好**：一个 PR 只解决一个问题，几百行的 PR 比几千行的 PR 审查快得多
- **写清楚动机**：正文说清楚「什么问题、你的方案、如何验证」，最好附上效果截图
- **保持礼貌**：开源社区都是志愿者，友好的语气很重要

8.5 本章小任务

- [] 自己开两个仓库账号不方便的话，就用 8.2 的流程：在分支上开发 → 发 PR → 自己审查自己 → Squash and merge → 清理分支，完整走一遍
- [] 找一个小的开源项目（比如某个教程仓库），发现一个错别字，按 8.3 的 Fork 流程发起你人生第一个对外 PR
- [] 在 PR 的 Files changed 里给某行代码留一条评论，体验行内评论

下一章：GitHub 网页上还有哪些宝藏功能——Issues、Actions、Pages、搜索技巧。

第 9 章 GitHub 网页功能导览

本章目标：认识 GitHub 网页上除了「存代码」之外的实用功能，知道什么场景用什么。

9.1 仓库标签栏全景

打开任意仓库主页，顶部有一排标签，按重要性排序：

标签	作用	使用频率
Code	文件列表、克隆地址（默认页）	每天
Issues	问题跟踪：报 bug、提需求、讨论	经常
Pull requests	合并申请单列表	经常
Actions	自动化流水线（测试、部署）	看结果时
Projects	看板式任务管理	视需要
Wiki	项目文档	偶尔
Settings	仓库设置（改名、删除、分支保护等）	偶尔

9.2 Issues：报 bug 和提需求的地方

Issues 是仓库的「问题台账」，用来：

- **报 bug**：「登录按钮点了没反应」
- **提需求**：「希望支持深色模式」
- **发起讨论**：「要不要升级到 Vue 3？」

创建一个 Issue：Issues 标签 → **New issue** → 填写标题和正文 → **Submit new issue**。

好 Issue 的四要素（照着写，别人才能帮你）：

要素	示例
现象	点击登录按钮，页面无任何反应
复现步骤	1. 打开登录页 2. 输入账号密码 3. 点击登录
期望结果	跳转到个人主页
环境	Windows 11, Chrome 120, 项目版本 v1.2.0

Issue 与代码联动：

- 在提交信息或 PR 描述里写 `fix #12`，PR 合并后 **#12 号 Issue 自动关闭**

- 在 Issue 里用 @用户名 可以提醒某个人来看

9.3 Actions: 自动化流水线

Actions 是 GitHub 内置的**自动化机器人**：每当触发条件满足（比如有人 push），就自动执行你写好的任务。

典型用途：

场景	说明
自动测试	每次 push 自动跑测试，PR 里显示红叉/绿勾
自动部署	代码推到 main 就自动发布网站
定时任务	每天凌晨自动抓数据、发报告
自动构建	打包成安装包 / Docker 镜像

怎么用：在仓库里建一个文件 `.github/workflows/任意名字.yml`，写入任务定义。例如「每次 push 跑 Python 测试」：

```
name: 自动测试
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest
```

提交后：

- 每次 push 自动触发，结果在 **Actions** 标签查看
- 每个 PR 底部会显示一个 **✓ All checks have passed** 或红色 ×
- 失败时 GitHub 会给你**发邮件通知**

新手现在知道「它是干什么的」就够。进阶玩法：配合分支保护规则「测试不通过就不允许合并」，见 9.7。

9.4 GitHub Pages: 免费建一个网站

GitHub 免费送每个账号**静态网站托管**服务，地址形如 `https://你的用户名.github.io/仓库名/`。

能放什么：纯 HTML/CSS/JS、Vue/React 打包后的静态文件、Markdown 博客等（不能放需要后端的动态网站）。

启用步骤：

- 1. 准备一个仓库，里面放 `index.html`（网站首页）
- 2. 仓库 → **Settings** → 左侧 **Pages**
- 3. **Build and deployment** → **Source** 选 `Deploy from a branch`
- 4. **Branch** 选 `main`，目录选 `/ (root)`，点 **Save**
- 5. 等 1~2 分钟，访问给出的网址即可看到你的网站

每个人还有一个专属空间：创建一个名为 `你的用户名.github.io` 的仓库，它就是你的个人主页，地址就是 `https://你的用户名.github.io/`。

9.5 搜索：GitHub 是一座宝库

顶部搜索框输入关键词，按分类浏览结果。进阶筛选语法（直接敲在搜索框里）：

语法	含义
<code>python in:name</code>	名字里含 python
<code>stars:>1000</code>	星标数大于 1000
<code>language:typescript</code>	指定语言
<code>pushed:>2026-01-01</code>	最近仍在更新
<code>awesome in:name</code>	找「awesome 合集」——各领域的资源清单

`awesome-python`、`awesome-cpp`、`awesome-chatgpt-prompts`任何领域都有对应的 awesome 仓库，是**入行找学习资源最快的入口**。

9.6 Star / Watch / Fork 的区别

按钮	作用	类比
Star	收藏，表示喜欢/支持	点赞 + 书签
Watch	订阅通知，仓库有动态会通知你	关注
Fork	复制一份到自己名下	复印一份带走

9.7 分支保护规则（进阶但很实用）

防止自己（或队友）手滑把坏代码直接推进 main：

1. 仓库 → **Settings** → **Branches** → **Add branch protection rule**（或 Add classic branch protection rule）
2. **Branch name pattern** 填 `main`
3. 勾选：
 - **Require a pull request before merging**：必须走 PR 才能合入 main
 - **Require status checks to pass before merging**：Actions 测试通过才允许合并
4. 保存

从此所有改动都必须经过「PR + 审查 + 测试通过」才能进 main——这是**企业级项目的标准做法**，个人项目也强烈建议开启。

9.8 把 GitHub 当作品集

对求职者来说，GitHub 主页是**免费的简历**：

- **个人主页装修**：创建一个名为 `你的用户名` 的仓库（与用户名同名），里面的 README 会直接显示在你的 GitHub 个人主页上。写上简介、技能、项目链接、联系方式
- **仓库的 README 认真写**：面试官点进仓库第一眼看到的就是它
- **给仓库配 License**：开源项目建议选 MIT（最宽松常用）；纯个人练习可选不公开
- **提交历史规律**：连续的小绿点（Contributions 日历）能体现持续学习的习惯

9.9 不想用命令行？两个官方工具

工具	适合	获取
GitHub Desktop	怕命令行的纯新手，图形界面完成 add/commit/push	https://desktop.github.com
GitHub CLI (gh)	喜欢命令行的进阶用户，在终端直接建仓库、发 PR	winget install --id GitHub.cli 后运行 <code>gh auth login</code>

建议：即使有图形工具，也**先按本书把命令行流程走熟**——命令行是 Git 的「原版界面」，几乎所有教程、报错解答都以它为准。图形工具只是把同样的命令包装成按钮。

9.10 本章小任务

- [] 给自己的一个仓库创建一条 Issue（比如记录一个想做的功能）
- [] 给仓库开一个 GitHub Pages，看到自己的网页上线

- [] 用搜索语法 `stars:>1000 language:python` 找到 3 个感兴趣的仓库并 Star
- [] 创建一个与用户名同名的仓库，写一份个人主页 README

下一章：附录——命令速查表和常见报错对照表，出问题先翻它。

第 10 章 附录：命令速查与常见问题

10.1 命令速查表

一次性配置（每台电脑只做一次）

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
ssh-keygen -t ed25519 -C "你的邮箱" # 生成 SSH 密钥（公钥添加到 GitHub）
```

新项目上传（第 4 章）

```
git init # 初始化
git add . # 暂存所有文件
git commit -m "首次提交" # 提交
git branch -M main # 分支改名 main
git remote add origin git@github.com:用户名/仓库名.git
git push -u origin main # 推送（-u 后日常只需 git push）
```

日常循环（第 5 章）

```
git pull # 开工拉取
git status # 看状态
git diff # 看具体改动
git add . # 暂存
git commit -m "类型：说明" # 提交
git push # 推送
git log --oneline --graph # 看历史
```

后悔药（第 6 章）

```
git restore --staged 文件 # 取消暂存
git restore 文件 # 丢弃工作区改动 ⚠️
git commit --amend -m "新信息" # 改最近提交信息
git reset --soft HEAD~1 # 撤销最近提交，保留改动
git reset --hard 提交ID # 回到历史版本 ⚠️
git revert 提交ID # 反向抵消已推送的提交
```

分支 (第 7 章)

```
git branch                # 查看分支
git switch 分支名         # 切换
git switch -c 分支名      # 创建并切换
git merge 分支名          # 合并到当前分支
git branch -d 分支名      # 删除分支
git push origin --delete 分支名 # 删除远程分支
git merge --abort         # 取消合并（冲突时）
```

其他常用

```
git stash                # 暂存改动（切走干别的）
git stash pop            # 取回暂存的改动
git remote -v            # 查看远程地址
git clone git@github.com:用户名/仓库名.git # 克隆仓库
git tag v1.0.0 && git push --tags # 打版本标签
```

10.2 常见报错对照表

报错信息	原因	解决办法
<code>fatal: not a git repository</code>	在非仓库文件夹里执行了 Git 命令	先 <code>cd</code> 到项目文件夹，或右键文件夹选 Git Bash Here
<code>fatal: unable to access ...</code> <code>Failed to connect</code>	网络问题 / 代理问题	检查网络；公司网络可能屏蔽，尝试手机热点测试；有代理时检查 <code>git config --global http.proxy</code>
<code>Permission denied</code> <code>(publickey)</code>	SSH 密钥没配好	按第 2 章 2.4 节重配；或改用 HTTPS 地址 + Personal Access Token
<code>! [rejected] main -> main</code> <code>(fetch first)</code>	云端有本地没有的提交	<code>git pull --rebase</code> 后再 <code>push</code> ；有冲突先解决
<code>! [rejected] main -> main</code> <code>(non-fast-forward)</code>	同上，历史分叉了	同上；单人仓库可 <code>git pull</code> 后合并再 <code>push</code>
<code>error: remote origin already exists</code>	重复添加远程地址	忽略它，直接 <code>push</code> ；或 <code>git remote set-url origin</code> 新地址
<code>Please tell me who you are</code>	没配置用户名邮箱	执行 2.3 节的两条 <code>config</code> 命令
<code>Everything up-to-date</code>	没有新东西要推	不是错误，云端已是最新
<code>warning: LF will be replaced by CRLF</code>	Windows 换行符警告	无碍；执行 <code>git config --global core.autocrlf true</code> 消除
提交后陷入奇怪的编辑界面	<code>commit</code> 忘了加 <code>-m</code> ，进入了 Vim	按 <code>Esc</code> ，输入 <code>:q!</code> 回车退出，重新带 <code>-m</code> 提交

报错信息	原因	解决办法
Your branch is ahead of origin/main by 1 commit	本地有提交没推送	执行 <code>git push</code> 即可
Support for password authentication was removed	用 HTTPS + 密码推送	GitHub 已禁用密码，改用 SSH（推荐）或 Personal Access Token

10.3 .gitignore: 让 Git 忽略该忽略的文件

有些文件**永远不该上传**:

- 依赖包目录 (`node_modules/`、`__pycache__`) —— 体积巨大且可重新安装
- 密钥文件 (`.env`、`*.key`) —— **泄露等于裸奔**，这是安全红线
- 系统垃圾 (`Thumbs.db`、`.DS_Store`)、IDE 配置 (`.vscode/`、`.idea/`)

用法: 在仓库根目录创建 `.gitignore` 文件，每行一条规则:

```
# 注释用 # 开头

# 依赖目录
node_modules/
__pycache__/

# 环境变量和密钥（千万不要提交！）
.env
*.key

# 系统垃圾
.DS_Store
Thumbs.db

# 编辑器配置
.vscode/
.idea/
```

创建时机: **最好在第一次 `git add .` 之前**。如果已经提交过再添加 `.gitignore`，需要先 `git rm -r --cached` 要忽略的目录 清除跟踪记录。

已有现成模板: 建仓库时在 Add `.gitignore` 下拉框里选择你的语言 (Python、Node 等)，GitHub 自动生成一套合适的规则。

10.4 敏感信息泄露怎么办

如果你已经把密钥、密码提交并推送到了云端——**光删除文件远远不够，历史记录里还有**。正确做法:

1. **立即作废该密钥**: 去对应平台 (如阿里云、AWS) 吊销并重新生成，这是第一优先级

- 2. 如果仓库是私有的且没有别人看过：最简单是**删除整个仓库重建**（把本地敏感文件清掉后重新推）
- 3. 如果是公开仓库：用 `git filter-repo` 清洗全部历史，并知会协作者

原则：**默认「一旦推到云端就视为泄露」**。宁可本地多检查一步 `git status`，也不要赌「私有仓库应该没人看」。

10.5 HTTPS 方式的备用方案（SSH 配不好时用）

公司网络有时屏蔽 SSH 的 22 端口，可以退回 HTTPS：

```
git clone https://github.com/用户名/仓库名.git
```

但 HTTPS 推送需要「密码」——现在要用 **Personal Access Token（个人访问令牌）** 代替：

- 1. GitHub → 头像 → Settings → 最底部 **Developer settings**
- 2. **Personal access tokens** → **Tokens (classic)** → **Generate new token**
- 3. 勾选 `repo` 权限，生成后**立即复制保存**（只显示一次）
- 4. 推送时用户名填 GitHub 用户名，密码粘贴这个 token

10.6 进阶学习资源

资源	地址	说明
GitHub 官方文档	https://docs.github.com	最权威，有中文版
交互式 Git 分支练习	https://learngitbranching.js.org	图形化闯关，强烈推荐
Pro Git 中文版	https://git-scm.com/book/zh/v2	系统学习 Git 的经典教材
GitHub Skills	https://skills.github.com	GitHub 官方的交互式入门课
Oh Shit, Git!?!	https://ohshitgit.com	各种「搞砸了怎么办」的急救手册

10.7 最后的建议

- 1. **从小开始**：先一个人用 Git 一个月，再谈协作；先用好 `add/commit/push`，再学分支
- 2. **提交要勤**：宁可一天提交十次小改动，不要一周提交一次大杂烩
- 3. **push 前先 status + diff**：养成自查习惯，能避免 90% 的事故
- 4. **别怕搞砸**：本地仓库的一切都有后悔药（第 6 章）；真正不可逆的只有「把密钥推到公开仓库」
- 5. **多用 GitHub 搜**：你遇到的几乎所有问题，别人都在 Issues 里问过、解决过

恭喜读完！现在打开 Git Bash，敲下 `git status`，开始你的第一个项目吧。